

Django Performance & You



Presented by @st3v3nmw (Software Engineer)

Agenda

0. Quick walkthrough on the Demo Project & Django Silk
1. The N+1 Query Problem
2. Interpreting EXPLAIN ANALYZE
3. Scan Types
4. Covering & Partial Indexes
5. Periodic Tasks

[Link to code repository.](#)

Profile Before Optimizing

Premature optimization is the root of all evil ~ Donald Knuth

The problem with premature optimization is that you never know in advance where the bottlenecks will be while coding the system.

A heuristic / rule of thumb to address this would be:

1. Make it work
2. Make it right
3. Use the system and find performance bottlenecks
4. Use a profiler in those bottlenecks to determine what needs to be optimized
5. Make it fast

<https://wiki.c2.com/?ProfileBeforeOptimizing=>

Performance Targets

Ideally, backend APIs should target a `p50` latency of `100 - 150ms`, and a `p95` latency of `250 - 300ms`. Some systems which will remain unnamed have tail latencies (`p99s`, `p99.9s`) greater than `20,000ms` 🦴. Tail latencies affect your most valuable customers since they have more data, etc.

Django Silk Setup

In `settings.py`, add the following:

```
1  INSTALLED_APPS = (  
2      ...  
3      "silk"  
4  )  
5  
6  MIDDLEWARE = [  
7      "silk.middleware.SilkyMiddleware",  
8      ...  
9  ]  
10  SILKY_PYTHON_PROFILER = True  
11  SILKY_ANALYZE_QUERIES = True  
12  SILKY_PYTHON_PROFILER_BINARY = True  
13  SILKY_PYTHON_PROFILER_RESULT_PATH = "profiling/"  
14  SILKY_EXPLAIN_FLAGS = {"costs": True, "verbose": True}
```

To enable access to the user interface add the following to your `urls.py`:

```
1  urlpatterns += [ ("silk/", include("silk.urls", namespace="silk")) ]
```

Then run `migrate` & `collectstatic`.

Example 1: The N+1 Query Problem

http://127.0.0.1:8000/api/messages/list_some/

🕒 2,794ms overall 📡 484ms on queries 📊 103 queries

```
1 class MessageViewSet(ReadOnlyModelViewSet):
2     """Message viewset."""
3
4     @action(methods=["GET"], detail=False)
5     def list_some(self, request):
6         """List some messages."""
7         messages = models.Message.objects.all()[:100]
8         data = serializers.MessageSerializer(messages, many=True).data
9         return Response(data=data, status=HTTP_200_OK)
```

```
1 @action(methods=["GET"], detail=False)
2 def list_some(self, request):
3     """List some messages."""
4     messages = models.Message.objects.all()[:100]
5     data = []
6     for message in messages:
7         sent_by = message.sent_by
8         data.append({"sender": sent_by.full_name, "text": message.text})
9     return Response(data=data, status=HTTP_200_OK)
```



Example 1: Queries

From <http://127.0.0.1:8000/silk/requests/>

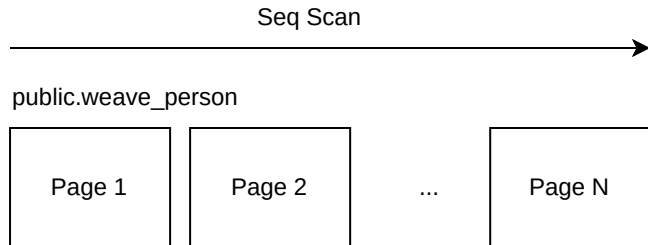
```
1  # Executed 1X -> Pick 100 messages
2  SELECT * FROM "weave_message" ORDER BY "weave_message"."updated" DESC LIMIT 100
3
4  # Executed 100X -> Once for each message
5  SELECT * FROM "weave_person" ORDER BY "weave_person"."updated" DESC
```

Query Plan 1

SELECT * FROM "weave_person" ORDER BY "weave_person"."updated" DESC

From <http://127.0.0.1:8000/silk/request/GUID/sql/ID/>

```
1  Sort (cost=73.83..76.33 rows=1000 width=76) (actual time=0.600..0.700 rows=1000 loops=1)
2    Output: id, created, updated, deleted, first_name, last_name, email_address
3    Sort Key: weave_person.updated DESC
4    Sort Method: quicksort  Memory: 141kB
5    -> Seq Scan on public.weave_person (cost=0.00..24.00 rows=1000 width=76) (actual time=0.013..0.207 rows=1000 loop
6      Output: id, created, updated, deleted, first_name, last_name, email_address
7    Planning Time: 0.081 ms
8    Execution Time: 0.815 ms
```



Example 1: Query Plan 2

From <http://127.0.0.1:8000/silk/request/GUID/sql/ID/>

SELECT * FROM "weave_message" ORDER BY "weave_message"."updated" DESC LIMIT 100

```
1  Limit (cost=5515.46..5526.96 rows=100 width=104) (actual time=27.490..30.617 rows=100 loops=1)
2    Output: id, created, updated, deleted, text, sent_by_id, thread_id
3    -> Gather Merge (cost=5515.46..12280.22 rows=58824 width=104) (actual time=27.489..30.607 rows=100 loops=1)
4      Output: id, created, updated, deleted, text, sent_by_id, thread_id
5      Workers Planned: 1
6      Workers Launched: 1
7      -> Sort (cost=4515.45..4662.51 rows=58824 width=104) (actual time=24.413..24.418 rows=70 loops=2)
8        Output: id, created, updated, deleted, text, sent_by_id, thread_id
9        Sort Key: weave_message.updated DESC
10       Sort Method: top-N heapsort  Memory: 61kB
11       Worker 0:  actual time=21.745..21.752 rows=100 loops=1
12       Sort Method: top-N heapsort  Memory: 62kB
13       -> Parallel Seq Scan on public.weave_message (cost=0.00..2267.24 rows=58824 width=104) (actual time=0
14         Output: id, created, updated, deleted, text, sent_by_id, thread_id
15         Worker 0:  actual time=0.008..6.095 rows=43056 loops=1
16  Planning Time: 0.093 ms
17  Execution Time: 30.644 ms
```


Example 1: Optimized

http://127.0.0.1:8000/api/messages/list_some/

 *1,487ms overall*  *82ms on queries*  *1 queries*

```
1 @action(methods=["GET"], detail=False)
2 def list_some(self, request):
3     """List some messages."""
4     messages = (
5         models.Message.objects
6         .select_related("sent_by").all()[0:100]
7     )
8     data = serializers.MessageSerializer(messages, many=True,).data
9     return Response(data=data, status=HTTP_200_OK)
```

Queries

From <http://127.0.0.1:8000/silk/request/GUID/sql/ID/>

```
1 # Executed 1X -> Picks everything
2 SELECT * FROM "weave_message"
3 INNER JOIN "weave_person" ON ("weave_message"."sent_by_id" = "weave_person"."id")
4 ORDER BY "weave_message"."updated" DESC
5 LIMIT 100
```

Example 1: Query Plan

```
1  Limit (cost=5707.03..5718.53 rows=100 width=180) (actual time=77.875..81.329 rows=100 loops=1)
2    Output: weave_message.id, weave_message.created, weave_message.updated, weave_message.deleted, weave_message.text,
3    -> Gather Merge (cost=5707.03..12471.79 rows=58824 width=180) (actual time=77.874..81.319 rows=100 loops=1)
4      Output: weave_message.id, weave_message.created, weave_message.updated, weave_message.deleted, weave_message.text,
5      -> Sort (cost=4707.02..4854.08 rows=58824 width=180) (actual time=73.949..73.957 rows=85 loops=2)
6        Output: weave_message.id, weave_message.created, weave_message.updated, weave_message.deleted, weave_message.text,
7        Sort Key: weave_message.updated DESC
8        Sort Method: top-N heapsort  Memory: 78kB
9        -> Hash Join (cost=36.50..2458.80 rows=58824 width=180) (actual time=0.687..36.358 rows=50000 loops=2)
10          Output: weave_message.id, weave_message.created, weave_message.updated, weave_message.deleted, weave_message.text,
11          Inner Unique: true
12          Hash Cond: (weave_message.sent_by_id = weave_person.id)
13          -> Parallel Seq Scan on public.weave_message (cost=0.00..2267.24 rows=58824 width=104) (actual time=0.000..0.000 rows=58824 loops=1)
14            Output: weave_message.id, weave_message.created, weave_message.updated, weave_message.deleted, weave_message.text,
15            -> Hash (cost=24.00..24.00 rows=1000 width=76) (actual time=0.591..0.591 rows=1000 loops=2)
16              Output: weave_person.id, weave_person.created, weave_person.updated, weave_person.deleted, weave_person.text,
17              Buckets: 1024  Batches: 1  Memory Usage: 114kB
18              -> Seq Scan on public.weave_person (cost=0.00..24.00 rows=1000 width=76) (actual time=0.000..0.000 rows=1000 loops=1)
19                Output: weave_person.id, weave_person.created, weave_person.updated, weave_person.deleted, weave_person.text,
20  Planning Time: 0.459 ms
21  Execution Time: 81.380 ms
```

Example 2: Unoptimized

<http://127.0.0.1:8000/api/threads/>

 113,450ms overall  27,349ms on queries  10,244 queries

```
1  class ThreadViewSet(ReadOnlyModelViewSet):
2
3      queryset = models.Thread.objects.filter(deleted=False)
4      serializer_class = serializers.ThreadSerializer
5
6  class MessageSerializer(serializers.ModelSerializer):
7
8      sender = serializers.ReadOnlyField(source="sent_by.full_name")
9
10     class Meta:
11         model = Message
12         fields = "__all__"
13
14  class ThreadSerializer(serializers.ModelSerializer):
15
16     messages = MessageSerializer(many=True)
17
18     class Meta:
19         model = Thread
20         fields = "__all__"
```

Example 2: Queries

```
1  # Executed 1X -> Picks 100 threads
2  SELECT * FROM "weave_thread"
3  WHERE NOT "weave_thread"."deleted"
4  ORDER BY "weave_thread"."updated" DESC
5  LIMIT 100
6
7  # Executed 100X -> Picks approx. 100 messages per thread
8  SELECT * FROM "weave_message"
9  WHERE "weave_message"."thread_id" = X
10 ORDER BY "weave_message"."updated" DESC
11
12 # Executed approx 10,000X -> Once for each message in each thread
13 SELECT * FROM "weave_person" WHERE "weave_person"."id" = X LIMIT 21
14
15 # Executed 1X -> Pagination
16 SELECT COUNT(*) AS "__count"
17 FROM "weave_thread"
18 WHERE NOT "weave_thread"."deleted"
19 # {
20 #   "count": 903,
21 #   "next": "http://127.0.0.1:8000/api/threads/?page=2",
22 #   "previous": null,
23 #   "results": [ ],
24 # }
```

Example 2: After prefetching related messages

<http://127.0.0.1:8000/api/threads/>

🕒 135,790ms overall ⚙️ 34,587ms on queries ~~123~~ 10,145 queries

```
1 class ThreadViewSet(ReadOnlyModelViewSet):
2     queryset = (
3         models.Thread.objects
4         .filter(deleted=False)
5         .prefetch_related("messages")
6     )
```

Queries

```
1 # Executed 1X -> Picks 100 threads
2 SELECT * FROM "weave_thread" WHERE NOT "weave_thread"."deleted" ORDER BY "weave_thread"."updated" DESC LIMIT 100
3
4 # Executed 1X -> Picks approx. 10,000 messages for all threads
5 SELECT * FROM "weave_message" WHERE "weave_message"."thread_id" IN (THREAD_IDS,)
6 ORDER BY "weave_message"."updated" DESC
7
8 # Executed approx 10,000X -> Once for each message in each thread
9 SELECT * FROM "weave_person" WHERE "weave_person"."id" = X LIMIT 21
10
11 SELECT COUNT(*) AS "__count" FROM "weave_thread" WHERE NOT "weave_thread"."deleted"
```

Example 2: After prefetching related senders 🤖

<http://127.0.0.1:8000/api/threads/>

🕒 7,183ms overall 📡 1,360ms on queries ~~123~~ 4 queries

```
1 class ThreadViewSet(ReadOnlyModelViewSet):
2     queryset = (
3         models.Thread.objects
4         .filter(deleted=False)
5         .prefetch_related("messages", "messages__sent_by")
6     )
```

Queries

```
1 SELECT * FROM "weave_thread" WHERE NOT "weave_thread"."deleted" ORDER BY "weave_thread"."updated" DESC LIMIT 100
2
3 # Executed 1X -> Picks approx. 10,000 messages for all threads
4 SELECT * FROM "weave_message" WHERE "weave_message"."thread_id" IN (THREAD_IDS,)
5 ORDER BY "weave_message"."updated" DESC
6
7 # Executed approx 1X -> Picks all relevant message senders
8 SELECT * FROM "weave_person" WHERE "weave_person"."id" IN (MESSAGE_IDS,)
9 ORDER BY "weave_person"."updated" DESC
10
11 SELECT COUNT(*) AS "__count" FROM "weave_thread" WHERE NOT "weave_thread"."deleted"
```

Example 3: Covering Indexes

<http://127.0.0.1:8000/api/messages/>

 88.364ms

```
1  # We will be focusing on only one query this time
2  # This query is run during pagination to get the total
3  # number of objects in the database
4
5  SELECT COUNT(*) AS "__count" FROM "weave_message"
6  WHERE NOT "weave_message"."deleted"
```

Query Plan

```
1  Aggregate (cost=2904.02..2904.03 rows=1 width=8) (actual time=23.799..23.800 rows=1 loops=1)
2    Output: count(*)
3    -> Seq Scan on public.weave_message (cost=0.00..2679.00 rows=9007 width=0) (actual time=0.008..17.689 rows=89964)
4        Output: id, created, updated, deleted, text, sent_by_id, thread_id
5        Filter: (NOT weave_message.deleted)
6        Rows Removed by Filter: 10036
7  Planning Time: 0.063 ms
8  Execution Time: 23.826 ms
```

Example 3: Covering Indexes

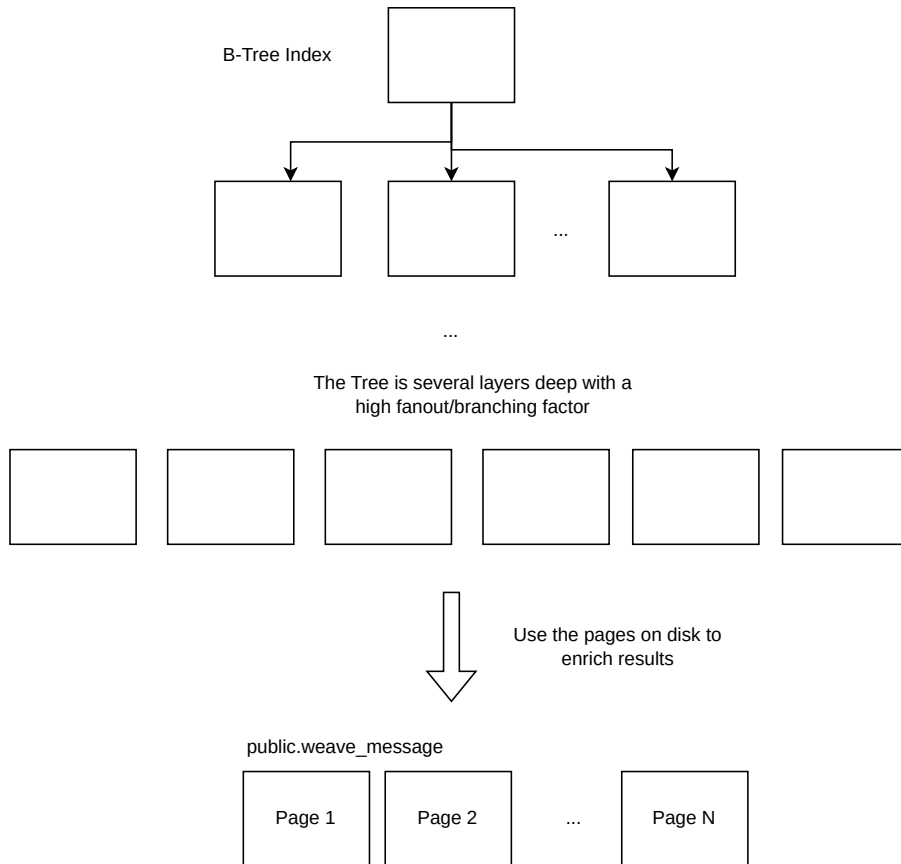
```
1 class AbstractBase(models.Model):
2     """Abstract Base."""
3
4     created = models.DateTimeField(auto_now_add=True)
5     updated = models.DateTimeField(auto_now=True)
6     deleted = models.BooleanField(db_index=True)
7
8     class Meta:
9         ordering = ("-updated",)
10        abstract = True
```

Query Plan

 14.985ms

```
1  Aggregate  (cost=2116.43..2116.44 rows=1 width=8) (actual time=17.813..17.814 rows=1 loops=1)
2    Output: count(*)
3    -> Index Only Scan using weave_message_deleted_9517d20b on public.weave_message (cost=0.29..1891.41 rows=90007 wi
4          Output: deleted
5          Index Cond: (weave_message.deleted = false)
6          Heap Fetches: 0
7  Planning Time: 0.078 ms
8  Execution Time: 17.846 ms
```


Example 3: Covering Indexes



Example 3: Partial Indexes

```
1 from django.db.models import Index, Q
2
3 class Message(AbstractBase):
4     class Meta(AbstractBase.Meta):
5         indexes = [
6             Index(
7                 fields=["deleted"],
8                 name="soft_deletes_ignore",
9                 condition=Q(deleted=False),
10            )
11        ]
```

Query Plan

 21.83ms

```
1 Aggregate (cost=1887.20..1887.21 rows=1 width=8) (actual time=25.893..25.895 rows=1 loops=1)
2   Output: count(*)
3   -> Index Only Scan using soft_deletes_ignore on public.weave_message (cost=0.29..1662.18 rows=90007 width=0) (acti
4     Output: deleted
5     Heap Fetches: 0
6 Planning Time: 0.137 ms
7 Execution Time: 25.931 ms
```

Example 4: Index Scans

```
1  > from threads.weave.models import *
2  > deleted_messages = Message.objects.filter(deleted=True)
3  > str(deleted_messages.query)
4  SELECT * FROM "weave_message" WHERE "weave_message"."deleted" ORDER BY "weave_message"."updated" DESC
5  > deleted_messages.explain(analyze=True) # Before Index
6  Sort (cost=3342.87..3367.85 rows=9993 width=104) (actual time=21.625..22.368 rows=10036 loops=1)
7    Sort Key: updated DESC
8    Sort Method: quicksort  Memory: 1835kB
9    -> Seq Scan on weave_message (cost=0.00..2679.00 rows=9993 width=104) (actual time=0.020..17.314 rows=10036 loops=1)
10      Filter: deleted
11      Rows Removed by Filter: 89964
12  Planning Time: 0.099 ms
13  Execution Time: 23.005 ms
14  > deleted_messages.explain(analyze=True) # After Index
15  Sort (cost=2556.54..2581.52 rows=9993 width=104) (actual time=16.202..17.342 rows=10036 loops=1)
16    Sort Key: updated DESC
17    Sort Method: quicksort  Memory: 1835kB
18    -> Bitmap Heap Scan on weave_message (cost=113.74..1892.67 rows=9993 width=104) (actual time=1.528..9.371 rows=10036 loops=1)
19      Filter: deleted
20      Heap Blocks: exact=1676
21      -> Bitmap Index Scan on weave_message_deleted_9517d20b (cost=0.00..111.24 rows=9993 width=0) (actual time=0.000..0.000 rows=9993 loops=1)
22        Index Cond: (deleted = true)
23  Planning Time: 0.207 ms
24  Execution Time: 18.260 ms
```

SELECT COUNT(*) Estimates

Do we really need exact total counts during pagination? 🤔

Google Search Doesn't...

"About 25,200,000,000 results (0.45 seconds)"

```
1  # threads.weave.paginators.py
2
3  class CustomPaginator(Paginator):
4
5      @cached_property
6      def count(self) -> int:
7          """Return the total number of objects, across all pages."""
8          with connection.cursor() as cursor:
9              cursor.execute(
10                  "SELECT reltuples FROM pg_class WHERE relname = %s",
11                  [self.object_list.query.model._meta.db_table],
12              )
13              return int(cursor.fetchone()[0])
14
15  class CustomPageNumberPagination(PageNumberPagination):
16
17      django_paginator_class = CustomPaginator
```

SELECT COUNT(*) Estimates

☰ 1.39ms

Query

```
1  SELECT reltuples FROM pg_class WHERE relname = weave_message
2  # Output = 100,000; Correct = 89,964 🙄
3  # Filters/conditions not applied
```

The catalog `pg_class` catalogs tables and most everything else that has columns or is otherwise similar to a table. This includes indexes, views, materialized views, composite types, and TOAST tables; see `relkind`. When we mean all of these kinds of objects we speak of “relations”.

`reltuples` is the number of live rows in the table. This is only an estimate used by the planner. It's updated by `VACUUM`, `ANALYZE`, and a few DDL commands such as `CREATE INDEX`.

~ <https://www.postgresql.org/docs/current/catalog-pg-class.html>

Query Plan

```
1  Index Scan using pg_class_relname_nsp_index on pg_class
2    (cost=0.28..8.29 rows=1 width=4)
3    (actual time=0.043..0.046 rows=1 loops=1)
```

Are we able to surface the count estimates from the query planner? `EXPLAIN` only, yes.

SELECT COUNT(*) Estimates

Introducing `TABLESAMPLE SYSTEM/BERNOULLI (n)` 🥁

🟢 11.50ms

But can we do better? Yes!

The `TABLESAMPLE SYSTEM/BERNOULLI` method returns an approximate percentage of rows. It generates a random number for each physical storage page for the underlying relation. Based on this random number and the sampling percentage specified, it either includes or exclude the corresponding storage page. If that page is included, the whole page will be returned in the result set.

~ https://wiki.postgresql.org/wiki/TABLESAMPLE_Implementation

```
1  SELECT COUNT(*) AS "__count" FROM "weave_message" TABLESAMPLE BERNOULLI (10) REPEATABLE (42)
2  WHERE NOT "weave_message"."deleted"
3  # Average output from several runs = 90,122.5; Correct = 89,964
4  # Off by ONLY 0.18%
```

Query Plan

```
1  Aggregate (cost=1801.50..1801.51 rows=1 width=8) (actual time=11.505..11.506 rows=1 loops=1)
2  # Original time taken: Around 25ms
```

SELECT COUNT(*) Estimates

```
1 class CustomPaginator(Paginator):
2     @cached_property
3     def count(self) -> int:
4         """Return an estimate of the total number of objects (* VERY * hacky btw).
5         Could be made more robust by first running the normal COUNT(*)
6         with a `SET LOCAL statement_timeout TO 50`
7         and then reverting to this if that doesn't complete in time.
8         """
9         with connection.cursor() as cursor:
10             db_table = self.object_list.model._meta.db_table
11             query = (
12                 self.object_list.annotate(__count=Count("*"))
13                 .values("__count")
14                 .order_by()
15                 .query
16             )
17             query.group_by = None
18             sql, params = query.get_compiler(DEFAULT_DB_ALIAS).as_sql()
19             sql = sql.replace(
20                 f'FROM "{db_table}"',
21                 f'FROM "{db_table}" TABLESAMPLE BERNOULLI (10) REPEATABLE (42)',
22             )
23             cursor.execute(sql, params)
24             return 10 * int(cursor.fetchone()[0])
```

Periodic Tasks

- Periodic tasks are one of the 4 horsemen of the apocalypse ☹️. They should be avoided like the plague.
 - Instead, prefer event-driven tasks
 - These type tasks run immediately after an event e.g. `instance.save`, `instance.create`, `instance.transition`, Django signals, etc,...
 - You can use Celery to set-up automatic retries for certain errors
 - Retry with exponential backoff to reduce load
 - For reporting, separate your transactional (Postgres) and analytical (BigQuery) databases so that reports don't slow down your main database
- If you must:
 - Do things in bulk e.g. `bulk_create`, `bulk_update`
 - Use iterators when looping over a large number of objects (`qs.iterator()`)
 - Helps reduce memory usage
 - Add indexes to fields that are used to filter. This speeds up reads but slows down writes (there's no such thing as a free lunch)

Periodic Tasks

```
1 q = Entry.objects.filter(headline__startswith="What")
2 q = q.filter(pub_date__lte=datetime.date.today())
3 q = q.exclude(body_text__icontains="food")
4 print(q)
```

- Realize that queriesets are lazy, so avoid things like:

- ``len(qs)`` -> ``qs.count()``
- ``list(qs)`` ☹
- ``bool(qs)`` -> ``qs.exists()``

QuerySets are lazy – the act of creating a QuerySet doesn't involve any database activity. You can stack filters together all day long, and Django won't actually run the query until the QuerySet is evaluated

<https://docs.djangoproject.com/en/4.2/ref/models/queriesets/#when-queriesets-are-evaluated>

Thank You!

And if everything fails, try API caching lol 🤪